

## Seminar 11

# Metoda Simplex

Asist. univ. dr. Mihai Tudor

5 mai 2026

# 1 Aspecte teoretice

Considerăm problema de programare liniară în forma canonică:

$$\min c^T x \quad \text{s.t. } Ax = b, x \geq 0, \quad (1)$$

unde  $A \in \mathbb{R}^{m \times n}$ ,  $b \in \mathbb{R}^m$ ,  $c \in \mathbb{R}^n$ ,  $n > m$ .

**Observație 1.1.** Orice problemă cu constrângeri  $Ax \leq b$ ,  $x \geq 0$  se aduce la forma canonică prin introducerea **variabilelor de slack (variabilelor de "abateri")**  $x_{n+i} \geq 0$ :

$$Ax + x_{\text{slack}} = b, \quad x_{\text{slack}} \geq 0.$$

Dacă  $b \geq 0$ , baza inițială fezabilă este formată chiar din variabilele de slack.

**Definiție 1.1.** O **soluție de bază acceptabilă** este un vector  $x = (x_1, \dots, x_m, 0, \dots, 0)$  în care primele  $m$  componente corespund unei baze  $\{A^1, \dots, A^m\}$  de coloane liniar independente ale lui  $A$ , astfel încât  $x_i \geq 0$ . Soluția se numește **nedegenerată** dacă  $x_i > 0$  pentru  $1 \leq i \leq m$ .

Metoda Simplex pornește cu o soluție de bază acceptabilă și trece la alta care face să descrească funcția obiectiv, până când este atinsă optimalitatea.

## 2 Tabloul Simplex

Presupunem că avem o soluție de bază fezabilă  $x = (x_1, x_2, \dots, x_m, 0, \dots, 0)$ , adică:

$$x_1 A^1 + x_2 A^2 + \dots + x_m A^m = b. \quad (2)$$

Orice vector  $A^q$  cu  $q > m$  poate fi scris în reprezentarea bazei curente ca:

$$A^q = y_{1q} A^1 + y_{2q} A^2 + \dots + y_{mq} A^m. \quad (3)$$

**Tabloul Simplex** asociat bazei  $\{A^1, \dots, A^m\}$  este:

| $A^1$    | $A^2$    | $A^3$    | $\dots$ | $A^m$    | $A^{m+1}$   | $A^{m+2}$   | $\dots$ | $A^n$    | $b$      |
|----------|----------|----------|---------|----------|-------------|-------------|---------|----------|----------|
| 1        | 0        | 0        | $\dots$ | 0        | $y_{1,m+1}$ | $y_{1,m+2}$ | $\dots$ | $y_{1n}$ | $y_{10}$ |
| 0        | 1        | 0        | $\dots$ | 0        | $y_{2,m+1}$ | $y_{2,m+2}$ | $\dots$ | $y_{2n}$ | $y_{20}$ |
| 0        | 0        | 1        | $\dots$ | 0        | $y_{3,m+1}$ | $y_{3,m+2}$ | $\dots$ | $y_{3n}$ | $y_{30}$ |
| $\vdots$ | $\vdots$ | $\vdots$ |         | $\vdots$ | $\vdots$    | $\vdots$    |         | $\vdots$ | $\vdots$ |
| 0        | 0        | 0        | $\dots$ | 1        | $y_{m,m+1}$ | $y_{m,m+2}$ | $\dots$ | $y_{mn}$ | $y_{m0}$ |

(4)

Soluția curentă este  $x_i = y_{i0}$  pentru  $1 \leq i \leq m$  și  $x_j = 0$  pentru  $j > m$ .

**Costurile reduse**  $r_j = c_j - z_j$

Notăm  $c_B = (c_1, \dots, c_m)^T$  vectorul costurilor variabilelor de bază. Valoarea curentă a funcției obiectiv este:

$$z_0 = c_B^T y_{\cdot 0} = \sum_{i=1}^m c_i y_{i0}.$$

Pentru fiecare variabilă nebazică  $j \in \{m+1, \dots, n\}$ , definim:

$$z_j = y_{1j}c_1 + \dots + y_{mj}c_m = \sum_{i=1}^m y_{ij}c_i, \quad (5)$$

și **costul redus** (costul relativ):

$$r_j := c_j - z_j, \quad j \in \{m+1, \dots, n\}.$$

Funcția obiectiv se poate scrie:

$$z = z_0 + (c_{m+1} - z_{m+1})x_{m+1} + \dots + (c_n - z_n)x_n = z_0 + \sum_{j=m+1}^n r_j x_j. \quad (6)$$

### 3 Pașii iterației Simplex

#### Pasul 1: Alegerea coloanei pivot (variabila care intră în bază)

Se caută  $q \in \{m+1, \dots, n\}$  cu  $r_q < 0$ , de obicei cel mai negativ cost redus:

$$q = \arg \min_{j > m} r_j.$$

Dacă  $r_j \geq 0$  pentru toți  $j$ , soluția curentă este **optimă**.

#### Pasul 2: Alegerea liniei pivot (variabila care iese din bază)

Se calculează rapoartele:

$$\varepsilon = \min_{1 \leq i \leq m} \left\{ \frac{y_{i0}}{y_{iq}}, \quad y_{iq} > 0 \right\}. \quad (7)$$

Minimul se atinge pentru indicele  $i_0$ : variabila  $A^{i_0}$  iese din bază,  $A^q$  intră.

Dacă nu există  $y_{iq} > 0$ , mulțimea de soluții fezabile este nemărginită.

#### Pasul 3: Pivotarea (actualizarea tabloului)

Elementul pivot este  $y_{i_0 q}$ .

(a) Împărțim rândul  $i_0$  la  $y_{i_0 q}$ .

(b) Pentru fiecare  $i \neq i_0$ :  $\text{Rând}_i \leftarrow \text{Rând}_i - y_{iq} \cdot \text{Rând}_{i_0}$ .

(c) Actualizăm baza:  $A^{i_0} \leftarrow A^q$ .

### 4 Exemplu complet pas cu pas

**Exemplu 4.1.** Rezolvăm prin metoda Simplex:

$$\begin{aligned} \max \quad & 3x_1 + 2x_2 \quad \Leftrightarrow \quad \min \quad -3x_1 - 2x_2 \\ \text{s.t.} \quad & x_1 + x_2 \leq 4, \quad x_1 \leq 3, \quad x_2 \leq 3, \quad x_1, x_2 \geq 0. \end{aligned}$$

**Forma canonică** (introducem variabilele de slack  $x_3, x_4, x_5 \geq 0$ ):

$$x_1 + x_2 + x_3 = 4, \quad x_1 + x_4 = 3, \quad x_2 + x_5 = 3.$$

$c = (-3, -2, 0, 0, 0)^T$ , baza inițială  $\{A^3, A^4, A^5\}$ , soluție inițială  $x = (0, 0, 4, 3, 3)$ .

**Tabloul inițial**

| Bază              | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ | $b$       |
|-------------------|-------|-------|-------|-------|-------|-----------|
| $A^3$             | 1     | 1     | 1     | 0     | 0     | 4         |
| $A^4$             | 1     | 0     | 0     | 1     | 0     | 3         |
| $A^5$             | 0     | 1     | 0     | 0     | 1     | 3         |
| $r_j = c_j - z_j$ | -3    | -2    | 0     | 0     | 0     | $z_0 = 0$ |

$c_B = (0, 0, 0)$ , deci  $z_j = 0$  și  $r_j = c_j$  pentru toți  $j$ .

**Iterația 1**

**Coloana pivot:**  $r_1 = -3 < r_2 = -2$ , deci  $q = 1$  ( $x_1$  intră în bază).

**Linia pivot** (regula  $\varepsilon$ , relația 7):

$$\varepsilon = \min \left\{ \frac{4}{1}, \frac{3}{1} \right\} = 3, \quad \text{la } i_0 = 2.$$

$A^4$  iese din bază,  $A^1$  ( $x_1$ ) intră. Pivotul este  $y_{21} = 1$ .

Operații elementare pe rânduri ( $R_i \leftarrow R_i - y_{i1} \cdot R_2$ ,  $R_2$  rămâne neschimbat):

$$R_1 \leftarrow R_1 - 1 \cdot R_2, \quad R_3 \leftarrow R_3 - 0 \cdot R_2.$$

| Bază  | $x_1$    | $x_2$ | $x_3$ | $x_4$ | $x_5$ | $b$        |
|-------|----------|-------|-------|-------|-------|------------|
| $A^3$ | <b>0</b> | 1     | 1     | -1    | 0     | 1          |
| $A^1$ | <b>1</b> | 0     | 0     | 1     | 0     | 3          |
| $A^5$ | <b>0</b> | 1     | 0     | 0     | 1     | 3          |
| $r_j$ | 0        | -2    | 0     | 3     | 0     | $z_0 = -9$ |

Soluție curentă:  $x_1 = 3$ ,  $x_2 = 0$ ,  $x_3 = 1$ ,  $z_0 = -9$  (adică  $\max = 9$ ).

Calculăm  $r_2 = c_2 - z_2 = -2 - (c_B^T \cdot y_2)$ . Cu  $c_B = (0, -3, 0)$  și coloana  $x_2 = (1, 0, 1)^T$ :

$$z_2 = 0 \cdot 1 + (-3) \cdot 0 + 0 \cdot 1 = 0, \quad r_2 = -2 - 0 = -2.$$

**Iterația 2**

**Coloana pivot:** singurul  $r_j < 0$  este  $r_2 = -2$ , deci  $q = 2$  ( $x_2$  intră în bază).

**Linia pivot:**

$$\varepsilon = \min \left\{ \frac{1}{1}, \frac{3}{1} \right\} = 1, \quad \text{la } i_0 = 1.$$

$A^3$  iese,  $A^2$  ( $x_2$ ) intră. Pivotul este  $y_{12} = 1$ .

Operații:  $R_3 \leftarrow R_3 - 1 \cdot R_1$ ,  $R_2$  neschimbat.

| Bază  | $x_1$ | $x_2$    | $x_3$ | $x_4$ | $x_5$ | $b$         |
|-------|-------|----------|-------|-------|-------|-------------|
| $A^2$ | 0     | <b>1</b> | 1     | -1    | 0     | 1           |
| $A^1$ | 1     | <b>0</b> | 0     | 1     | 0     | 3           |
| $A^5$ | 0     | <b>0</b> | -1    | 1     | 1     | 2           |
| $r_j$ | 0     | 0        | 2     | 1     | 0     | $z_0 = -11$ |

**Condiție de optimalitate:**  $r_j \geq 0$  pentru toți  $j$  (Teorema 3.7.2).

**Soluție optimă:**  $x_1 = 3$ ,  $x_2 = 1$ ,  $x_3 = 0$ ,  $x_4 = 0$ ,  $x_5 = 2$ .

$$z^* = -z_0 = 11 \quad (\text{pentru maximizare}).$$

## 5 Implementare Python — Metoda Simplex

### 5.1 Implementarea tabloului Simplex

```

1 import numpy as np
2
3 def simplex(c, A, b, afiseaza=True):
4     """
5     Metoda Simplex pentru minimizarea  $c^T x$ 
6     cu restrictiile  $Ax \leq b, x \geq 0$ .
7
8     Parametri:
9         c -- vectorul costurilor (lungime n)
10        A -- matricea restrictiilor (m x n)
11        b -- vectorul drept (lungime m); trebuie b  $\geq 0$ 
12        afiseaza -- daca True, afiseaza tabloul la fiecare iteratie
13
14    Returneaza:
15        x_opt -- solutia optima
16        z_opt -- valoarea optima a functiei obiectiv
17        stare -- 'optim', 'nemarginit' sau 'infezabil'
18    """
19    m, n = A.shape
20    n_var = n # numarul variabilelor originale
21
22    # Adaugam variabile de slack: tabloul are m linii si n+m+1
23    # coloane
24    # [A | I_m | b]
25    tab = np.zeros((m, n + m + 1))
26    tab[:, :n] = A # matricea originala
27    tab[:, n:n+m] = np.eye(m) # variabilele de slack
28    tab[:, -1] = b # membrul drept
29
30    # Baza initiala: variabilele de slack (indicii n, n+1, ..., n
31    # +m-1)
32    baza = list(range(n, n + m))
33
34    # Vectorul c extins cu zerouri pentru variabilele de slack
35    c_ext = np.concatenate([c, np.zeros(m)])
36
37    def afiseaza_tablou(iteratie):
38        """Afiseaza tabloul Simplex curent."""
39        n_total = n + m
40        print(f"\n{'='*60}")
41        print(f"    Iteratia {iteratie}")
42        print(f"{'='*60}")
43        # Header
44        header = "Baza |"
45        for j in range(n_total):
46            header += f"    x{j+1:2d} "

```

```

46     print(header)
47     print("-" * len(header))
48     for i in range(m):
49         linie = f"    x{baza[i]+1:2d} |"
50         for j in range(n_total):
51             linie += f" {tab[i,j]:5.2f}"
52         linie += f" | {tab[i,-1]:5.2f}"
53         print(linie)
54     # Costuri reduce
55     rj = calculeaza_rj()
56     print("-" * len(header))
57     linie_r = "    rj |"
58     for j in range(n_total):
59         linie_r += f" {rj[j]:5.2f}"
60     z0 = np.dot(c_ext[baza], tab[:, -1])
61     linie_r += f" | {z0:5.2f}"
62     print(linie_r)
63
64     def calculeaza_rj():
65         """Calculeaza costurile reduce rj = cj - zj."""
66         n_total = n + m
67         cb = c_ext[baza]
68         rj = np.zeros(n_total)
69         for j in range(n_total):
70             if j in baza:
71                 rj[j] = 0.0
72             else:
73                 zj = np.dot(cb, tab[:, j]) # zj = sum_i yij*ci
74                 rj[j] = c_ext[j] - zj # rj = cj - zj
75         return rj
76
77     if afiseaza:
78         afiseaza_tablou(0)
79
80     for iteratie in range(1, 100):
81         rj = calculeaza_rj()
82
83         # Pasul 1: Alegem coloana pivot q (cel mai negativ rj)
84         q = np.argmin(rj)
85         if rj[q] >= -1e-10:
86             # Toti rj >= 0 => solutie optima (Teorema 3.7.2)
87             x_opt = np.zeros(n + m)
88             for i, b_idx in enumerate(baza):
89                 x_opt[b_idx] = tab[i, -1]
90             z_opt = np.dot(c_ext[baza], tab[:, -1])
91             if afiseaza:
92                 print(f"\n>>> Conditia de optimalitate: rj >= 0
93                     pentru toti j")
94                 print(f">>> Solutie optima gasita!")
95             return x_opt[:n], z_opt, 'optim'

```

```

96      # Pasul 2: Alegem linia pivot i0 (regula epsilon, rel.
97          3.17)
98      col_pivot = tab[:, q]
99      yi0 = tab[:, -1]
100
101      rapoarte = []
102      for i in range(m):
103          if col_pivot[i] > 1e-10: # yiq > 0
104              rapoarte.append((yi0[i] / col_pivot[i], i))
105
106      if not rapoarte:
107          # Nicio linie pivot => multimea fezabila e
108          # nemrginita
109          if afiseaza:
110              print(f"\n>>> Problema nem rginita: yiq <= 0
111                  pentru toti i")
112              return None, -np.inf, 'nemarginit'
113
114      # epsilon = min { yi0/yiq : yiq > 0 }
115      epsilon, i0 = min(rapoarte, key=lambda x: x[0])
116
117      if afiseaza:
118          print(f"\n Coloana pivot: q={q+1} (x{q+1}), rq={rj[q]
119                  :.4f}")
120          print(f" Rapoarte yi0/yiq: "
121                  + ", ".join(f"{r:.3f}(i={i+1})" for r,i in
122                      rapoarte))
123          print(f" Linia pivot: i0={i0+1}, epsilon={epsilon:.4
124                  f}")
125          print(f" Iese x{baza[i0]+1}, intra x{q+1}")
126
127      # Pasul 3: Pivotare (operatii elementare pe r nduri)
128      pivot = tab[i0, q]
129      tab[i0, :] /= pivot # impartim randul pivot la elementul
130                          # pivot
131
132      for i in range(m):
133          if i != i0:
134              tab[i, :] -= tab[i, q] * tab[i0, :] # eliminare
135
136      # Actualizam baza
137      baza[i0] = q
138
139      if afiseaza:
140          afiseaza_tablou(iteratie)
141
142      return None, None, 'nedeterminat'

```

## 5.2 Funcții ajutătoare

```

1 import numpy as np
2
3 def afiseaza_solutie(x_opt, z_opt, stare, c, tip='min'):
4     """
5     Afiseaza solutia problemei de programare liniara.
6     tip -- 'min' sau 'max'
7     """
8     print(f"\n{'='*50}")
9     print(f"    REZULTAT: {stare.upper()}")
10    print(f"{'='*50}")
11    if stare == 'optim':
12        for j, xj in enumerate(x_opt):
13            print(f"    x{j+1} = {xj:.6f}")
14            if tip == 'max':
15                print(f"    Valoare maxima: z* = {-z_opt:.6f}")
16            else:
17                print(f"    Valoare minima: z* = {z_opt:.6f}")
18    elif stare == 'nemarginit':
19        print("    Functia obiectiv este nem rginita.")
20    else:
21        print(f"    Problema {stare}.")
22
23
24 def verifica_solutie(x_opt, c, A, b, tip='min'):
25     """
26     Verifica fezabilitatea si valoarea obiectivului solutiei.
27     """
28     print("\n--- Verificare ---")
29     # Valoarea functiei obiectiv
30     val = np.dot(c, x_opt)
31     print(f"    c^T x = {val:.6f}    ({'min' if tip=='min' else 'max'
32         '})")
33     # Constrangerile
34     Ax = A @ x_opt
35     print(f"    Constrangeri Ax <= b:")
36     for i in range(len(b)):
37         ok = Ax[i] <= b[i] + 1e-8
38         print(f"        {Ax[i]:.4f} <= {b[i]}    {'OK' if ok else '
39             VIOLAT'}")
40     # Nenegativitate
41     neg = np.any(x_opt < -1e-8)
42     print(f"    x >= 0: {'OK' if not neg else 'VIOLAT'}")

```

### 5.3 Aplicarea pe exemplul din curs

```

1 import numpy as np
2
3 # Problema: max 3x1 + 2x2 => min -3x1 - 2x2
4 # s.t. x1+x2 <= 4, x1 <= 3, x2 <= 3, x1,x2 >= 0
5

```

```

6  c = np.array([-3.0, -2.0])    # negat pentru minimizare
7
8  A = np.array([
9      [1.0, 1.0],    # x1+x2 <= 4
10     [1.0, 0.0],    # x1 <= 3
11     [0.0, 1.0],    # x2 <= 3
12 ])
13 b = np.array([4.0, 3.0, 3.0])
14
15 print("=== Metoda Simplex aplicata ===")
16 print(f"Problema: max 3x1 + 2x2")
17 print(f"  x1+x2 <= 4, x1 <= 3, x2 <= 3, x1,x2 >= 0\n")
18
19 x_opt, z_opt, stare = simplex(c, A, b, afiseaza=True)
20 afiseaza_solutie(x_opt, z_opt, stare, c, tip='max')
21 verifica_solutie(x_opt, c, A, b, tip='max')
22
23 # Output:
24 # Iteratia 0 (tabloul initial):
25 # Baza /   x1   x2   x3   x4   x5 /   b
26 #   x3 /   1.00 1.00 1.00 0.00 0.00 /   4.00
27 #   x4 /   1.00 0.00 0.00 1.00 0.00 /   3.00
28 #   x5 /   0.00 1.00 0.00 0.00 1.00 /   3.00
29 #   rj /  -3.00 -2.00 0.00 0.00 0.00 /   0.00
30 #
31 # Coloana pivot: q=1 (x1), rq=-3.0000
32 # Linia pivot: i0=2, epsilon=3.0000
33 # Iese x4, intra x1
34 #
35 # [... tabloul dupa iteratia 1 ...]
36 # [... tabloul dupa iteratia 2 ...]
37 #
38 # Conditia de optimalitate: rj >= 0 pentru toti j
39 # Solutie optima gasita!
40 # x1 = 3.000000
41 # x2 = 1.000000
42 # Valoare maxima: z* = 11.000000

```

## 6 Condiții de optimalitate și caz nemărginit

**Teoremă 6.1** (Condiție de optimalitate, Teorema 3.7.2 din curs). *Dacă pentru o soluție de bază fezabilă  $(x_1, \dots, x_m, 0, \dots, 0)$  avem:*

$$r_j = c_j - z_j \geq 0 \quad \forall j \in \{m+1, \dots, n\},$$

*atunci acea soluție este **optimă**.*

**Teoremă 6.2** (Caz nemărginit, Teorema 3.7.1 din curs). *Dacă  $r_q < 0$  pentru un indice  $q$ , dar  $y_{iq} \leq 0$  pentru toți  $i \in \{1, \dots, m\}$ , atunci mulțimea de soluții acceptabile este **nemărginită** și funcția obiectiv poate lua orice valoare mică (spre  $-\infty$ ).*

```

1 import numpy as np
2
3 # Exemplu de problema nem rginita:
4 # min -x1 s.t. x1 - x2 <= 1, x1, x2 >= 0
5 # (nu exista marginire superioara pentru x1)
6
7 c_nem = np.array([-1.0, 0.0])
8 A_nem = np.array([[1.0, -1.0]])
9 b_nem = np.array([1.0])
10
11 print("=== Problema nem rginita ===")
12 x, z, stare = simplex(c_nem, A_nem, b_nem, afiseaza=True)
13 # Output: Problema nem rginita: yiq <= 0 pentru toti i
14
15 # Verificare cu scipy
16 from scipy.optimize import linprog
17 res = linprog(c_nem, A_ub=A_nem, b_ub=b_nem,
18              bounds=[(0, None)]*2, method='highs')
19 print(f"scipy status: {res.message}")
20 # scipy: Optimization failed. The problem is (trivially)
    unbounded.

```

## 7 Găsirea soluției de bază inițiale: metoda variabilelor artificiale

```

1 import numpy as np
2 from scipy.optimize import linprog
3
4 def simplex_doua_faze(c, A_eq, b_eq):
5     """
6     Metoda Simplex cu doua faze pentru probleme cu restrictii de
7     egalitate.
8
9     Faza I: gasim o solutie de baza fezabila (var. artificiale)
10    Faza II: optimizam functia obiectiv originala
11    """
12    m, n = A_eq.shape
13
14    # --- Faza I: minimizam suma variabilelor artificiale ---
15    # Variabile: [x (n), y_art (m)]
16    c_faza1 = np.concatenate([np.zeros(n), np.ones(m)])
17    A_faza1 = np.hstack([A_eq, np.eye(m)])
18    b_faza1 = b_eq.copy()
19
20    print("=== FAZA I: Gasirea solutiei initiale fezabile ===")
21    res1 = linprog(c_faza1, A_eq=A_faza1, b_eq=b_faza1,
22                  bounds=[(0, None)]*(n+m), method='highs')

```

```

23     if res1.fun > 1e-8:
24         print("Problema INFEZABILA (min var. artificiale > 0)")
25         return None, None, 'infezabil'
26
27     x_faza1 = res1.x[:n]
28     print(f"    Solutie initiala fezabila: x = {x_faza1}")
29     print(f"    Suma var. artificiale = {res1.fun:.2e}")
30
31     # --- Faza II: optimizam cu functia obiectiv originala ---
32     print("\n=== FAZA II: Optimizare ===")
33     res2 = linprog(c, A_eq=A_eq, b_eq=b_eq,
34                   bounds=[(0, None)]*n, method='highs')
35
36     print(f"    Status: {res2.message}")
37     return res2.x, res2.fun, 'optim'
38
39
40 # Exemplu: problema transportului (restrictii de egalitate)
41 c_t = np.array([2, 3, 1, 5, 4, 8], dtype=float)
42 A_t = np.array([
43     [1,1,1,0,0,0],
44     [0,0,0,1,1,1],
45     [1,0,0,1,0,0],
46     [0,1,0,0,1,0],
47     [0,0,1,0,0,1],
48 ], dtype=float)
49 b_t = np.array([300,400,200,250,250], dtype=float)
50
51 x_t, z_t, stare_t = simplex_doua_faze(c_t, A_t, b_t)
52 if stare_t == 'optim':
53     print(f"\nPlan transport optim: {x_t}")
54     print(f"Cost minim: {z_t:.0f} lei")

```

## 8 Comparatie: Simplex propriu vs. scipy

```

1  import numpy as np
2  from scipy.optimize import linprog
3  import time
4
5  def benchmark_simplex(probleme):
6      """
7      Compara implementarea proprie cu scipy.optimize.linprog.
8      """
9      print(f"{'Problema':>15} | {'Simplex propriu':>18} | "
10           f"{'scipy (HiGHS)':>16} | {'Acord':>8}")
11      print("-" * 65)
12
13      for nume, (c, A, b) in probleme.items():
14          c_np = np.array(c, dtype=float)
15          A_np = np.array(A, dtype=float)

```

```

16     b_np = np.array(b, dtype=float)
17
18     # Simplex propriu
19     x1, z1, st1 = simplex(c_np, A_np, b_np, afiseaza=False)
20
21     # scipy HiGHS
22     res = linprog(c_np, A_ub=A_np, b_ub=b_np,
23                 bounds=[(0, None)]*len(c), method='highs')
24     z2 = res.fun if res.success else None
25
26     acord = (z1 is not None and z2 is not None and
27             abs(z1 - z2) < 1e-6)
28     z1_str = f"{z1:.4f}" if z1 is not None else "---"
29     z2_str = f"{z2:.4f}" if z2 is not None else "---"
30     print(f"{nume:>15} | {z1_str:>18} | {z2_str:>16} | "
31           f"'DA' if acord else 'NU':>8}")
32
33
34 probleme_test = {
35     "2D (max)": (
36         [-3, -2],
37         [[1, 1], [1, 0], [0, 1]],
38         [4, 3, 3]
39     ),
40     "Dieta": (
41         [1.0, 1.5, 2.0],
42         [[-200, -150, -100], [-4, -8, -12], [-20, -200, -30]],
43         [-1800, -60, -800]
44     ),
45     "Silo": (
46         [-120, -90, -80],
47         [[1, 1, 1], [1, 0, 0], [0, 1, 0], [0, 0, 1], [-1, -1, 0]],
48         [1000, 500, 600, 400, -400]
49     ),
50 }
51
52 benchmark_simplex(probleme_test)

```

## 9 Exerciții propuse

1. Aplicați **manual** metoda Simplex (construiți tabloul și urmăriți iterațiile) pentru:

$$\max 5x_1 + 4x_2 \quad \text{s.t.} \quad 6x_1 + 4x_2 \leq 24, \quad x_1 + 2x_2 \leq 6, \quad x_1, x_2 \geq 0.$$

Identificați variabilele de slack, tabloul inițial, coloana și linia pivot la fiecare iterație și condiția de optimalitate.

2. Verificați cu funcția **simplex** din seminar că obțineți același rezultat ca la exercițiul 1. Afișați tabloul la fiecare iterație.

3. Adăugați la funcția **simplex** o coloană suplimentară care urmărește  $z_0$  la fiecare iterație și afișați evoluția valorii funcției obiectiv. Verificați că  $z_0$  este monoton descrescătoare (pentru minimizare).
4. Construiți un exemplu de problemă PL cu **soluții optime multiple** (funcția obiectiv paralelă cu o față a regiunii fezabile). Cum se observă această situație în tabloul Simplex? (*Indiciu*: unul dintre costurile reduse  $r_j = 0$  la optim pentru o variabilă nebazică.)
5. Modificați funcția **simplex** pentru a returna și **vectorul dual**  $\lambda^* = c_B^T B^{-1}$  (cunoscut și ca prețurile umbră). Interpretați economic valorile  $\lambda_i^*$ : ce reprezintă o creștere cu o unitate a lui  $b_i$ ?
6. Aplicați metoda cu **două faze** (`simplex_doua_faze`) pentru:

$$\min x_1 + 2x_2 \quad \text{s.t. } x_1 + x_2 = 5, \quad 2x_1 + x_2 = 7, \quad x_1, x_2 \geq 0.$$

Identificați soluția de bază fezabilă inițială găsită în Faza I.

7. **Analiză de sensibilitate**: pentru problema din exemplul principal ( $\max 3x_1 + 2x_2$ ), variați pe rând  $b_1 \in \{2, 3, 4, 5, 6\}$  și observați cum se modifică soluția optimă și valoarea funcției obiectiv. Trasați grafic  $z^*(b_1)$ .